

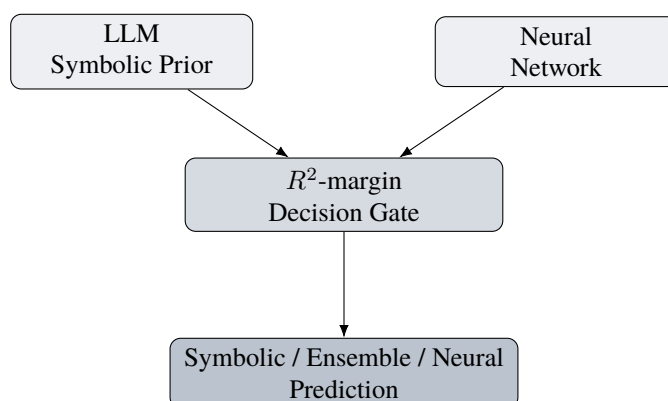
HypatiaX Hybrid Methods

A Static-Analysis Report on the LLM + Neural-Network Hybrid Symbolic Regression Pipeline

Reproducibility repository: <https://github.com/sednabcn/LLM-HypatiaX-REPRO>

Directories analysed: `hypatiaux/core/`, `hypatiaux/experiments/`,
`hypatiaux/tools/` and `hypatiaux/protocols/`

Revision 2 — extends the original `core/+experiments/` analysis with
`hypatiaux/tools/` (symbolic engines, template regressor, four-layer validator)



Contents

1	Scope and Method	2
2	a) Sources	2
2.1	Primary source	2
2.2	Source files examined (hybrid method implementations)	2
2.3	New in this revision: <code>hypatiaux/tools/</code>	3
2.4	New in this revision: <code>hypatiaux/experiments/benchmarks/</code> additions	4
2.5	New in this revision: <code>hypatiaux/protocols/</code> (context only)	4
3	b) Description	4
4	c) Algorithm (extracted)	6
4.1	Core hybrid decision loop (Variant 1, [C1])	6
4.2	Decision gate (Variant 2, [C2])	6
4.3	Uncertainty-weighted ensembling (Variant 3, [C3])	7
4.4	Symbolic-parameter refitting (<code>fit_formula_params</code> , [C1])	7
4.5	LLM-prior warm start for symbolic regression ([E1])	7
4.6	<code>SymbolicEngineWithLLM</code> 's four PySR modes ([T2])	7
4.7	A five-version-long dead hybridisation path ([T1] bug history)	8
4.8	Four-layer expression validation ([T5] -- [T8])	8
4.9	Benchmark-side orchestration	9
5	d) Advantages and Limitations	9
5.1	Advantages	9
5.2	Limitations	10
6	e) Update – What Changed in This Revision	11
7	f) Refactorization	12
7.1	Consolidate the duplicated API helper	12
7.2	Unify the ensemble-weighting strategies	13
7.3	Extract the shared <code>hybrid_predict</code> skeleton	13
7.4	Centralise magic numbers following the <code>adaptive_config.py</code> pattern	13
7.5	Promote ground-truth-leakage checks from comments to code	13
7.6	Isolate LLM-generated code execution	13
7.7	Compose the validation pipeline into the decision gate, not just alongside it	13
7.8	Record the engine version in its own output, not just its docstring	14
7.9	Reduce experiment-script duplication via composition, not copy-paste	14
8	Summary	14

1 Scope and Method

This report analyses the **hybrid symbolic-neural discovery methods** implemented in the LLM-HypatiaX-REPRO repository. It is a **revision** of an earlier report that covered only `hypatiax/core/` (the reusable method implementations) and `hypatiax/experiments/` (the benchmark and protocol scripts that exercise them); that revision reported *four* hybridisation variants (§ “b) Description”). This revision re-fetches the repository at master in full (via a tarball of the entire tree, not individual file guesses) and additionally reads `hypatiax/tools/` and `hypatiax/experiments/benchmarks/`, per the request that prompted this update. One correction to the request as stated: the directory does *not* exist at `hypatiax/core/tools/` – it is `hypatiax/tools/`, a package *sibling* of `core/`, not nested inside it (confirmed against the live repository tree). A further, previously-undocumented top-level package, `hypatiax/protocols/`, was also found and is summarised below for completeness, though it holds benchmark *test-case data* rather than hybridisation logic.

Cross-checking line-for-line against the previous report, every file previously read under `core/` ([C1]–[C6]) is **byte-identical in line count** to what was reported before (1,418 / 1,090 / 267 / 1,804 / 713+872 / 328 lines respectively) – the core hybrid-decision logic analysed previously (Variants 1–3, § “c) Algorithm”) has **not changed**. What has changed is everything *around* it: a substantial, previously unexamined `hypatiax/tools/` package (four symbolic-generation/regression engines, a four-layer expression-validation pipeline, and a PCA-split utility) and an expanded `experiments/benchmarks/` directory (PCA-directed OOD variants of the DeFi benchmark, an extracted extrapolation-ladder module, and a self-audit notebook that documents a genuine, still-open reliability discrepancy – § “f) Update” below). All line counts, function signatures, decision-logic branches and inline engineering notes (“FIX-*” comments) quoted below are taken verbatim from the source retrieved on September 20, 2026, not inferred.

The repository is a *reproducibility companion* for a paper referenced in its own docstrings as:

“HypatiaX: A Hybrid Symbolic-Neural Framework for Extrapolation-Reliable Analytical Discovery” (JMLR v3.0, Apr 2026)

as stated in `hypatiax/core/metrics.py`. The codebase therefore mixes (a) production-style method code, (b) CI/benchmark orchestration, and (c) a dense trail of bug-fix comments documenting the evolution of the hybrid decision logic across paper revisions – this trail is itself a primary source for Sections 4 and 6 below.

2 a) Sources

2.1 Primary source

- **Repository:** <https://github.com/sednabcn/LLM-HypatiaX-REPRO> (owner sednabcn, Apache-2.0 licence, “Original HypatiaX work © PhD Ruperto P. Bonet Chaple”).
- **Associated paper (as cited internally):** *HypatiaX: A Hybrid Symbolic-Neural Framework for Extrapolation-Reliable Analytical Discovery*, JMLR, v3.0 (Apr 2026) — cited in `hypatiax/core/metrics.py` and referenced by section number (e.g. §10.2, §10.6, §10.9) throughout `hypatiax/experiments/`.

2.2 Source files examined (hybrid method implementations)

- [C1] `hypatiax/core/generation/hybrid_defi_system/hybrid_system_nn_defi_domain.py` (1,418 lines) — class `EnhancedHybridSystemDeFi`: LLM-formula + scipy refit + NN, DeFi domain.
- [C2] `hypatiax/core/generation/hybrid_all_domains_llm_nn/hybrid_system_llm_nn_all_domains.py` (1,090 lines) — class `HybridSystemAllDomains`: same pattern generalised to physics/chemistry/biology (Feynman-style) domains.

- [C3] `hypatiax/core/generation/hybrid_defi_llm_nn/hybrid_ensemble_system_defi_domain.py` (267 lines) — worked example of *uncertainty-weighted* LLM/NN ensembling (`ensemble_llm_nn`).
- [C4] `hypatiax/core/base_pure_llm/baseline_pure_llm_defi_discovery.py` (1,804 lines) — the pure-LLM baseline (`PureLLMBaseline`) that both [C1] and [C2] delegate to for formula generation.
- [C5] `hypatiax/core/training/adaptive_config.py` (713 lines) and `baseline_neural_network_defi_improved.py` (872 lines) — a priori NN hyperparameter selection and the shared NN baseline architecture.
- [C6] `hypatiax/core/metrics.py` (328 lines) — shared R^2 /MSE/RMSE/MAE and paper-specific metrics (`compute_near_perfect_rate`, `compute_catastrophic_rate`).
- [E1] `hypatiax/experiments/benchmarks/hypatia.py` (432 lines) — `get_llm_prior()`: LLM-generated symbolic candidates used as a *warm start* for a genetic-programming symbolic regressor (PySR), a fourth, distinct hybridisation strategy.
- [E2] `hypatiax/experiments/benchmarks/exp1_ablation.py` (1,771 lines) — ablation of the LLM-prior contribution (§10.6/§10.9 of the paper); dynamically loads its driving engine from `tools/symbolic/hybrid` [T1] (see the version-drift note in §4).
- [E3] `hypatiax/experiments/benchmarks/run_hybrid_system_benchmark.py` (576), `run_comparative_suite_benchmark_v2.py` (5,492), `run_noise_sweep_benchmark.py` (964), `run_sample_complexity_benchmark.py` (1,131), `run_instability_suite.py` (1,438), `exp1_five_system.py` (518), `exp3_nguyen12_consolidated.py` (1,487) — orchestration/CI wrappers that instantiate [C1]/[C2] across DeFi, Feynman and Nguyen benchmark suites, with sharding, resume, and result-merging logic.
- [E4] `hypatiax/experiments/tests/test_enhanced_defi_extrapolation.py` (1,586 lines) — held-out extrapolation testing of the [C1] hybrid decision output. Four further test modules were found alongside it (`test_defi_benchmark_v4_select.py`, `test_defi_benchmark_v4_smoke.py`, `test_extrapolation_helpers.py`, `test_proc_box_wiring.py`) but were not read in full, being harness/smoke tests rather than method logic.

2.3 New in this revision: `hypatiax/tools/`

- [T1] `hypatiax/tools/symbolic/hybrid_system_v50_2.py` (1,586 lines) — class `HybridDiscoverySystem`, docstring-versioned **v5.4** (not v5.1 – see §4): an *orchestration wrapper* around [T2]’s `SymbolicEngineWithLLM` (constructed at line 427) that adds NaN/RMSE numerical-stability fixes, a `PhysicsAwareRegressor` [T3] fallback tier, and results-saving/statistics – it does *not* itself implement the LLM/PySR decision logic (correction to the previous revision of this report, which had misattributed that logic to this file; see §3/§4).
- [T2] `hypatiax/tools/symbolic/symbolic_engine.py` (3,235 lines) — `SymbolicEngine` / `SymbolicEngineWithLLM` (PySR-backed, “v21”), `BayesianRanker` (“v22”, re-ranks the PySR Pareto front by log-posterior rather than raw R^2), and `SymbolicTreeEngine` (“v23”, a self-contained expression-tree search with *no* PySR/Julia dependency) – three interchangeable discovery back-ends unified in one module. **`SymbolicEngineWithLLM` is the true implementer of the four LLM-integration modes** (`none/seed/hybrid/fallback`) that [E2] exercises via [T1], with its own genuine, independently-thresholded decision logic (§3.4) – making it a hybridisation variant in its own right, not merely an implementation detail of [T1]. This is confirmed by direct inspection: [T1] line 427 reads `self.symbolic_engine: SymbolicEngine = SymbolicEngineWithLLM(...)` and every LLM-mode branch dispatches into [T2]’s `discover()`, not into any LLM-mode logic of its own.
- [T3] `hypatiax/tools/symbolic/physics_aware_regressor.py` (2,117 lines) — class `PhysicsAwareRegressor`

(v11.1): a *template-library* candidate generator (Michaelis-Menten, competitive inhibition, Hill coefficients, Lineweaver-Burk, Bernoulli energy forms, ...) with train/validation-split early stopping and L_2 -regularised constant fitting; instantiated by [T1] as an explicit fallback path when the PySR search underperforms (line 729, “[FALLBACK] Using PhysicsAwareRegressor...”).

[T4] `hypatiax/tools/symbolic/smart_structure_detector.py` (620 lines) — classes `SmartStructureDetector` and `IntelligentEquationBuilder`: a *template-free* counterpart to [T3], inferring additive/multiplicative structure, per-variable term form and variable interactions directly from data statistics rather than from a template library.

[T5]–[T8] `hypatiax/tools/validation/symbolic_validator.py` (896), `dimensional_validator.py` (804), `domain_validator.py` (721), `ensemble_validator.py` (1,092) — a four-layer expression acceptance pipeline (Layer 3 syntax/SymPy → Layer 1 units via Pint → Layer 2 domain constraints → Layer 4 numerical stability), orchestrated by `EnsembleValidator` and detailed in §3.4.

[U1] `hypatiax/tools/utis/pca_split_utils.py` (120 lines) — `pca_directed_split()`: an alternative, PCA-projected train/test split used by the new PCA benchmark variants (§6).

2.4 New in this revision: `hypatiax/experiments/benchmarks/` additions

[E5] `hypatiax/experiments/benchmarks/extrap_r2_far.py` (500 lines) — `build_extrap_split()` and `compute_extrap_r2_far()`, extracted (per its own docstring, “*Extracted from run_comparative_suite_bench*” into a standalone module implementing a five-tier extrapolation quality ladder.

[E6] `hypatiax_defi_benchmark_v3.py` (2,322), `_v4.py` (2,639), and their PCA-split counterparts `_v3_pca.py` (2,246), `_v4_pca.py` (2,689) — the `_pca` variants replace the random train/test split with [U1]’s `pca_directed_split()` as the sole protocol split (`run_comparative_suite_benchmark_pca.py`, line 15: “*FIX-C3/DISCLOSURE: pca_directed_split is now the sole protocol split*”).

[E7] `portfolio_variance_v3c2.py` / `_v4c2.py` (991 lines each) — Colab-exported audit notebooks-as-scripts that reconcile a specific reliability discrepancy in the Portfolio Variance test case; discussed as a limitation in §4/§6.

2.5 New in this revision: `hypatiax/protocols/` (context only)

A fourth top-level package, not requested but encountered while resolving [E2]’s and [E5]’s imports, holds benchmark *test-case generators* rather than hybridisation logic: `experiment_protocol_all_30.py` (1,429, 30 multi-domain cases), `experiment_protocol_benchmark_v2.py` (1,794, the Feynman SR subset), `experiment_protocol_defi.py` (2,020, DeFi/finance cases), `experiment_protocol_nguyen12.py` (928, the Nguyen-12 suite). These are listed for completeness but not analysed further, since they define data rather than method.

All quoted code, comments and identifiers in this report are reproduced from these files as retrieved on September 20, 2026 from the master branch (fetched as a full-repository tarball via `code.load.github.com`, not individually-guessed paths).

3 b) Description

HypatiaX’s “hybrid method” is not a single algorithm but a **family of LLM+Neural-Network combination strategies**, all built around the same three ingredients:

1. an **LLM symbolic prior** – Claude (via the `anthropic` Python SDK) is prompted to derive a closed-form formula for a described physical/financial relationship and return it as an executable `def formula(...): block;`

2. a **neural-network baseline** – a small feed-forward MLP (ImprovedNN: Linear + LayerNorm + SiLU blocks, trained with AdamW and cosine-annealing-with-warm-restarts) fit directly to the numeric data, with no symbolic structure imposed;
3. a **combination/decision layer** that chooses, per case, whether to trust the symbolic formula, the network, or a blend of both, based on comparative R^2 on the training data.

Four concrete variants of this pattern exist in `core/experiments`:

Variant 1 – EnhancedHybridSystemDeFi [C1] Adds a *second symbolic-refinement stage*: after the LLM proposes a formula, its numeric constants are re-fit with `scipy.optimize.curve_fit(fit_formula_params)`, so the LLM only needs to get the *functional form* right, not the exact constants. The final choice among {fitted-symbolic, ensemble, NN} uses a fixed R^2 threshold plus a margin test.

Variant 2 – HybridSystemAllDomains [C2] Same two-model skeleton, generalised to physics/chemistry/biology “Feynman-style” equations, with a `force_llm` override that skips the NN entirely for domains where a power-law/exponential symbolic form is known a priori to dominate a linear-space MLP. Ties are broken in favour of the symbolic formula (`llm_r2 >= nn_r2`); a genuine NN win only triggers a weighted blend, never a silent NN-only fallback, unless the LLM failed outright.

Variant 3 – ensemble_llm_nn [C3] A worked example showing *uncertainty-weighted* (rather than R^2 -weighted) combination: each model’s weight is proportional to R_i^2/σ_i , where σ_i is the standard deviation of that model’s residuals – i.e. a precision-weighted ensemble, closer in spirit to inverse-variance weighting than to a hard decision gate.

Variant 4 – SymbolicEngineWithLLM’s four PySR modes [T2], exercised via [E1]/[T1] A fundamentally different hybridisation point from Variants 1–3: instead of blending two independently-trained predictors, the LLM and a genetic-programming symbolic regressor (PySR) take turns *proposing* and *refining* the same candidate expression. [E1]’s standalone `get_llm_prior()` illustrates the simplest form of this (LLM candidates as a PySR seed population); the actual implementer is the `SymbolicEngineWithLLM` class in [T2], which generalises it into *four* selectable modes chosen via an `llm_mode` parameter, each with its own decision logic (detailed in §3.4): `none` (pure PySR), `seed` (LLM hypothesis biases PySR’s operator set and warm-starts its population), `hybrid` (LLM proposes a formula first; PySR refines it *only* if the LLM’s R^2 falls in $[0.5, 0.95]$, with the better of the two kept), and `fallback` (PySR runs first; the LLM is invoked only if PySR’s $R^2 \leq 0.90$, again keeping whichever result is better). “Variant 4” is therefore itself a small family of four strategies with real, source-visible thresholds, not one fixed rule. It is *driven* by `HybridDiscoverySystem` [T1] (docstring-versioned v5.4, exercised by [E2]), which constructs a `SymbolicEngineWithLLM` instance and adds a physics-template fallback (Variant 5) on top, but the LLM/PySR decision logic itself lives in [T2], not [T1].

Variant 5 – Template-guided physics fallback [T3]/[T4] A hybridisation point below the LLM/NN layer entirely: when `HybridDiscoverySystem` [T1]’s PySR search underperforms, it falls back to `PhysicsAwareRegressor` [T3], which fits a *library* of domain templates (Michaelis-Menten, competitive inhibition, Bernoulli energy, ...) with regularised constant optimisation rather than searching expression space from scratch. `SmartStructureDetector` / `IntelligentEquationBuilder` [T4] implement the template-*free* counterpart – additive/multiplicative structure and per-variable term form are inferred statistically from the data rather than matched against a template library – so the two modules together offer both *prior-knowledge-driven* and *data-driven* routes to a symbolic candidate, independent of whether an LLM call is made at all.

All four variants share the same evaluation vocabulary defined in `hypatiax/core/metrics.py` [C6]: R^2 , RMSE, MAE, plus two paper-specific aggregate statistics – `compute_near_perfect_rate` (fraction of cases with $R^2 > 0.99$) and `compute_catastrophic_rate` (fraction with $R^2 < -1$, i.e. predictions worse than the mean baseline) – reflecting the paper’s stated concern with *extrapolation reliability* rather than

average in-distribution fit alone.

4 c) Algorithm (extracted)

4.1 Core hybrid decision loop (Variant 1, [c1])

The following pseudocode is a faithful extraction of `EnhancedHybridSystemDeFi.hybrid_predict()` (`hybrid_system_nn_defi_domain.py`, lines 1076–1262):

Algorithm 1 `hybrid_predict(description, domain,  $X_{train}$ ,  $y_{train}$ , vars, meta)`

```

1:  $result_{llm} \leftarrow \text{GENERATELLMFORMULA}(\text{description}, \text{domain}, \text{vars}, \text{meta})$ 
2:  $r2_{llm} \leftarrow 0$ 
3: if  $result_{llm}.\text{python\_code}$  is valid then
4:    $\hat{y}_{llm} \leftarrow \text{SAFEEXECFORMULA}(result_{llm}.\text{code}, X_{train})$ 
5:   if  $\hat{y}_{llm}$  finite then
6:      $r2_{llm} \leftarrow R^2(y_{train}, \hat{y}_{llm})$ 
7:   end if
8: end if
   Stage 2 — refit the LLM formula's numeric constants:
9:  $(code_{fit}, r2_{fit}) \leftarrow \text{FITFORMULAPARAMS}(result_{llm}.\text{code}, X_{train}, y_{train}, \text{vars})$ 
10: if  $r2_{fit} > r2_{llm} + 0.001$  then
11:   accept refit; else keep  $code_{fit} \leftarrow result_{llm}.\text{code}$ ,  $r2_{fit} \leftarrow r2_{llm}$ 
12: end if
   Stage 3 — train NN baseline (seed = sha256(description) mod  $2^{31}$ ):
13:  $(model_{nn}, \hat{y}_{nn}) \leftarrow \text{TRAINNNMODEL}(X_{train}, y_{train}, \text{seed})$ 
14:  $r2_{nn} \leftarrow R^2(y_{train}, \hat{y}_{nn})$ 
   Stage 4 — decision gate:
15:  $margin \leftarrow r2_{fit} - r2_{nn}$ 
16: if  $r2_{fit} \geq 0.85$  and  $margin > 0.05$  then
17:    $decision \leftarrow \text{fitted\_llm}$ ;  $\hat{y} \leftarrow \hat{y}_{fit}$ 
18: else if  $r2_{fit} \geq 0.5$  and  $r2_{nn} \geq 0.5$  and  $|margin| \leq 0.15$  then
19:    $w \leftarrow r2_{fit} / (r2_{fit} + r2_{nn})$ 
20:    $decision \leftarrow \text{ensemble}$ ;  $\hat{y} \leftarrow w \cdot \hat{y}_{fit} + (1 - w) \cdot \hat{y}_{nn}$ 
21: else
22:    $decision \leftarrow \text{nn}$ ;  $\hat{y} \leftarrow \hat{y}_{nn}$ 
23: end if
24: if  $\hat{y}$  not finite and  $\hat{y}_{nn}$  finite then
25:   fall back to nn
26: end if
27: return  $\{decision, \hat{y}, R^2(y_{train}, \hat{y}), model_{nn}, \dots\}$ 

```

4.2 Decision gate (Variant 2, [c2])

`HybridSystemAllDomains.hybrid_predict()` (lines 730–880) uses a *tie-favours-symbolic* rule instead of a fixed threshold:

```

1 if  $llm\_ok$  and  $llm\_r2 \geq nn\_r2$ :
2    $decision = "llm"$  # symbolic wins or ties
3 elif  $llm\_ok$  and  $nn\_r2 > llm\_r2$  and  $nn\_r2 > 0.90$ :
4   # both strong: R2-weighted blend of predictions
5    $w\_llm, w\_nn = \max(llm\_r2, 0), \max(nn\_r2, 0)$ 
6    $blend = (w\_llm * llm\_pred + w\_nn * nn\_pred) / (w\_llm + w\_nn)$ 
7    $decision = "ensemble"$ 
8 else:

```

```
9 decision = "nn" # LLM failed / R2<=0
```

Listing 1: Decision branch, `hybrid_system_llm_nn_all_domains.py` (abridged, original code)

A domain-level `force_llm` flag (injected by the benchmark runner for Feynman physics domains) short-circuits this entirely and returns the LLM result directly whenever $r2_{llm} > 0$, skipping NN training altogether – a deliberate domain prior encoded outside the model.

4.3 Uncertainty-weighted ensembling (Variant 3, [C3])

```
1 llm_unc = std(llm_pred - y_true); nn_unc = std(nn_pred - y_true)
2 w_llm = (1/(llm_unc+eps)) * max(llm_r2, 0)
3 w_nn = (1/(nn_unc+eps)) * max(nn_r2, 0)
4 w_llm, w_nn = w_llm/(w_llm+w_nn), w_nn/(w_llm+w_nn)
5 ensemble = w_llm*llm_pred + w_nn*nn_pred
```

Listing 2: `ensemble_llm_nn()`, `hybrid_ensemble_system_defi_domain.py` (original code, condensed)

This is a precision-weighted (inverse-residual-std) combination rather than a hard *select-one* decision, and is presented in the source as an illustrative alternative to Variants 1–2’s threshold logic rather than as the production path.

4.4 Symbolic-parameter refitting (`fit_formula_params`, [C1])

A supporting routine used by Variant 1’s Stage 2:

1. `_parametrize_formula()` rewrites the LLM’s Python formula so that its literal numeric constants become free parameters `_P[i]` – first by matching named assignments (`Vmax = 80.0` → `Vmax = _P[0]`), falling back to replacing *any* non-structural numeric literal ($\notin \{0, 1, 2\}$) if no named assignments are found;
2. `scipy.optimize.curve_fit` (with a differential_ evolution fallback for hard cases) then fits `_P` to (X_{train}, y_{train}) ;
3. the refit is accepted only if it improves training R^2 by more than 0.001 over the LLM’s original (unfitted) constants.

4.5 LLM-prior warm start for symbolic regression ([E1])

Algorithm 2 `get_llm_prior(eq,  $X_{train}$ ,  $y_{train}$ )` → PySR guesses

- 1: prompt ← `BUILD_PROMPT(eq.description, variable names, sample data)`
 - 2: response ← `Claude.messages.create(prompt, temperature = 0)`
 - 3: candidates ← `PARSE_EXPRESSIONS(response)` ▷ plain Python/numpy syntax, e.g. `x**3+x**2`
 - 4: **return** candidates ▷ caller must translate to its own PySR operator config before use
-

These candidates are then supplied to a genetic-programming symbolic regressor (PySR) as an initial *guess population*, so the LLM narrows the search space the evolutionary algorithm explores, rather than competing with it after the fact – the same idea generalised into [T2]’s *seed mode* (§4.6) and exercised throughout [E2] via [T1] (docstring-versioned v5.4, *not* v5.1 – §5, Limitations).

4.6 `SymbolicEngineWithLLM`’s four PySR modes ([T2])

Unlike Variants 1–3’s fixed R^2 /margin gates, [T2]’s `SymbolicEngineWithLLM.discover()` dispatches on an `llm_mode` parameter to one of four methods, each with its own, independently thresholded control flow (extracted directly from `_discover_with_llm_seed`, `_discover_hybrid` and `_discover_with_fallback`, lines 2035–2312):

- **none** – pure PySR genetic-programming search; the LLM plays no role at all.
- **seed** – an LLM hypothesis is generated first; its operators are extracted and unioned into PySR’s `extra_unary_/extra_binary_operators`, and the hypothesis’s own expression is passed as a `seed_expressions` warm start (only effective if the installed `pysr` build exposes a `guesses` argument).
- **hybrid** – the LLM proposes a formula first. If its $R^2 > 0.95$, that formula is returned directly and PySR never runs (`"hybrid_llm_only"`). If $R^2 < 0.5$, the LLM prior is discarded entirely and PySR runs cold (protecting PySR’s seed population from a bad LLM guess). Otherwise ($0.5 \leq R^2 \leq 0.95$) PySR is seeded/refined from the LLM candidate and the two results are compared, keeping whichever has the higher R^2 (`"hybrid_pysr_better"` vs. `"hybrid_llm_better"`).
- **fallback** – PySR runs first at full budget. If $R^2 > 0.90$ its result is returned immediately (`"fallback_pysr_only"`); only if PySR is at or below that threshold is the LLM invoked, and its hypothesis is kept only if it beats PySR’s R^2 (`"fallback_llm_better"` vs. `"fallback_pysr_better"/"fallback_both_failed"`).

HybridDiscoverySystem [T1] is the orchestrator that constructs a SymbolicEngineWithLLM with the requested mode and calls into it; when PySR/LLM together still underperform, [T1] falls further back to PhysicsAwareRegressor [T3] (Variant 5) rather than returning a failed result – so the production failure-handling path is a chain of progressively simpler hybridisation strategies (LLM-guided PySR, implemented in [T2] → template-matched regression, implemented in [T3]), spanning two files, not one decision point in one class.

4.7 A five-version-long dead hybridisation path ([T1] bug history)

[T1]’s own docstring records an unusually long-lived latent bug that is directly relevant to how much weight Variant 4’s benchmark history should be given: the `use_llm` parameter was *introduced* in v3.5, but SymbolicEngineWithLLM was never actually called – “*Bug: neither provider was ever called; SymbolicEngineWithLLM never used*” (v3.5), and this is explicitly reconfirmed as “*persisted*” across v3.8 and v4.1-PROD (“*self.anthropic_provider / self.google_provider set but never read; SymbolicEngineWithLLM never imported; use_llm flag never checked in method body*”) and v4.2/v4.2.1, before finally being repaired in v5.0’s “*LLM-wiring rewrite (FIX-1 ...FIX-6)*”. In other words: for five consecutive documented versions spanning the project’s own changelog, passing `use_llm=True` silently produced pure-PySR behaviour with no error and no warning. Any benchmark result recorded against a pre-v5.0 build under an ostensibly “hybrid” configuration would in fact reflect Variant 4 (none) rather than Variant 4 (seed/hybrid/fallback) – a concrete instance of the “documentation vs. behaviour” drift this report already flags for the v5.1/v5.4 engine label (§4), but considerably more consequential, since it affects what the numbers *measured* rather than merely how they were labelled.

4.8 Four-layer expression validation ([T5]–[T8])

Independent of which variant generated a candidate symbolic expression, EnsembleValidator [T8] composes four checks in a fixed order, each producing a 0–100 sub-score (paraphrased from the module docstrings):

1. **Symbolic** ([T5], weight 30%) – SymPy-based syntax and mathematical-pathology checks (undefined variables, NaN/complex infinity, division-by-zero risk, log/sqrt domain violations); supports both string and LaTeX input.
2. **Dimensional** ([T6], weight 30%) – Pint-based unit consistency across the expression tree; degrades rather than aborts on unit-parsing failures for individual variables, since PySR output may have partially-known variable sets.
3. **Domain** ([T7], weight 30%) – string/array-level constraint checks specific to the target domain

(DeFi, risk, finance, ESG): positivity, probability bounds, AMM constant-product and fee-bound invariants.

4. **Numerical** ([T8], weight 10%) – `lambdifys` the expression and evaluates it on held-out test data to catch NaN/Inf/overflow/underflow that the purely symbolic checks above cannot see.

A domain-aware reconciliation step after Layers 1–3 downgrades false-positive division-by-zero errors from the symbolic layer when domain constraints already guarantee the relevant variable is strictly positive (the module’s own example: Michaelis-Menten’s $K_m > 0$ by definition). The weighted score must clear **85.0/100** for the expression to be accepted. This pipeline sits *alongside* rather than *inside* Variants 1–3’s R^2 -based decision gates (§4 notes the gates themselves have no such acceptance threshold), so a formula can win the R^2 comparison in [C1]/[C2] and still be flagged as invalid by [T8] in a separate pass – the two mechanisms are not currently composed into one decision.

4.9 Benchmark-side orchestration

`run_hybrid_system_benchmark.py` [E3] chains together, per CI shard: (1) batch hybrid-system evaluation, (2) held-out extrapolation testing (73 cases, [E4]), (3) statistical/performance analysis, and (4) report generation, with resume-from-checkpoint support driven by `TASK_IDS/SHARD_IDS` environment variables forwarded to each subprocess.

5 d) Advantages and Limitations

5.1 Advantages

- **Complementary failure modes.** The LLM path supplies physically/financially interpretable closed-form structure (good extrapolation, poor fit when the true form is unknown to the model); the NN path fits arbitrary residual structure in-distribution but is known to extrapolate poorly. Blending them via an R^2 -based gate lets the system fall back gracefully rather than committing to either failure mode.
- **Two-stage symbolic refinement.** Separating *structure discovery* (LLM) from *constant fitting* (`scipy`) means the LLM does not need to produce numerically exact constants – only a plausible functional form – which is a substantially easier generation target and is explicitly designed to raise R^2 post-hoc (§“Stage 2” in [C1]).
- **Domain-aware overrides.** The `force_llm` mechanism in [C2] encodes a defensible prior (symbolic power-laws beat a linear-space MLP for known physics families) directly into the pipeline, avoiding wasted NN training and avoiding a documented regression (“previously flag was silently ignored... causing Snell’s/Zeeman regression”).
- **Explicit reproducibility engineering.** NN weight initialisation is deterministically seeded from `sha256(description)`; an optional `HYPATIA_X_LLM_FREEZE_CACHE` file makes LLM outputs bit-identical across separate process invocations, addressing the genuinely hard problem that the Anthropic API is not guaranteed to return byte-identical completions at temperature= 0 across requests – a limitation the code itself documents rather than silently ignores.
- **Numerically defensive execution.** `_safe_exec_formula` clips `exp/arcsin/arccos` domains, and every decision path has an explicit non-finite-value guard that falls back to the NN prediction, preventing a single bad LLM formula (e.g. $R^2 \approx -10^{78}$, observed and referenced in comments) from corrupting benchmark output.
- **Extrapolation-first evaluation.** Unlike a pure in-distribution R^2 average, `compute_near_perfect_rate / compute_catastrophic_rate` in [C6] and the dedicated 73-case extrapolation test [E4] directly

target the paper’s stated failure mode of interest (silent catastrophic extrapolation), which a single aggregate metric would hide.

- **Layered, independent expression validation.** [T5]–[T8] give the pipeline a domain-agnostic acceptance gate (syntax $\rightarrow$ units $\rightarrow$ domain constraints $\rightarrow$ numerical stability) that is decoupled from any one hybrid variant’s R^2 logic, so the same four-layer check can be reused whichever of Variants 1–5 produced the candidate expression.
- **PCA-directed out-of-distribution evaluation.** The `_pca` benchmark variants [E6] replace a random train/test split with `pca_directed_split()` [U1], which sorts samples along the first principal component and holds out the far end of that axis – a genuinely out-of-distribution test rather than a random subsample, complementing (not replacing) the multiplier-based “far” extrapolation ladder in [E5].

5.2 Limitations

- **Threshold logic is hand-tuned, not learned.** The $R^2 \geq 0.85/\text{margin} > 0.05$ (Variant 1) and $R^2 > 0.90$ (Variant 2) constants that govern the symbolic/ensemble/NN choice are fixed literals in the source with no accompanying sensitivity analysis in `core/`; small changes to these thresholds would change which examples are classified `fitted_llm` vs. `ensemble` vs. `nn` without any documented calibration procedure.
- **Decision made on training R^2 , not held-out data.** Both [C1]’s and [C2]’s `hybrid_predict` choose the final method using R^2 computed on (X_{train}, y_{train}) – the same data both models were fit on – so the gate itself has no protection against a model (particularly the NN) that overfits training data while extrapolating poorly; that protection is pushed entirely onto the separate extrapolation test suite [E4] rather than being part of the hybrid decision itself.
- **LLM non-determinism only partially solved.** The frozen-cache mechanism only removes cross-run variance *for repeated identical (description, domain, variables) keys*; a first-time query to the API is still subject to the documented “*server-side batching/kernel effect*” non-determinism at temperature= 0, which no amount of local seeding can fix.
- **Prompt-based ground-truth leakage was an active, recently-fixed bug class.** Multiple comments (FIX GT-LEAK, FIX GT-LEAK-2) describe prior prompt versions that embedded the expected answer or a near-complete pre-solved formula inside the LLM prompt itself, inflating reported accuracy; these are fixed in the current code but indicate the benchmark’s reported numbers have previously been vulnerable to this failure mode, and no automated leakage check is present in `core/` to prevent recurrence.
- **Code-execution-based evaluation.** `_safe_exec_formula` and `execute_python_code_get_predictions` both call Python’s `exec()` directly on LLM-generated text. Sandboxing is limited to a restricted namespace of numpy functions; there is no process isolation, resource limiting, or static validation of the LLM output before execution.
- **Substantial functional duplication across variants.** The API-call helper `_create_message_deterministic` (temperature fallback for models that deprecate the parameter) is *independently reimplemented* in [C1], [C2], and [C4] rather than imported from one place – an explicit, acknowledged design choice (“per-file copies rather than cross-module imports”), which trades a small amount of import coupling for a real risk of the three copies drifting out of sync under future edits (see §6).
- **Ensemble weighting is inconsistent across variants.** Variant 1 weights by raw R^2 share; Variant 2 weights by $\max(R^2, 0)$ share; Variant 3 weights by R^2/σ_{resid} . None of the three is derived from, or validated against, the others, so the “ensemble” concept is not a single reusable component but three separately-maintained formulas with different statistical justifications.

- **No uncertainty quantification on the LLM output itself.** The LLM is treated as a point-estimate formula generator; nothing in `core/` captures token-level or self-reported confidence, so “LLM R^2 ” is the only signal available to the decision gate, conflating *formula correctness* with *model confidence*.
- **Documented version drift between caller and engine.** The driving engine’s own docstring identifies it as **v5.4** ([T1]), listing five rounds of NaN-RMSE fixes (RC-1–RC-5) since v5.3 and v5.2; the caller [E2] that loads it still refers to it as “*HybridDiscoverySystem v5.1*” in its own docstring, engine label, and results dictionary (“engine”: “hybrid_system_v50_2.py” with no version field). This is a documentation-hygiene gap rather than a functional bug – the loader resolves the file dynamically by path, not by an asserted version – but it means result artefacts produced by [E2] do not self-report which of at least three engine revisions (v5.2/v5.3/v5.4) actually produced them.
- **Variant 4’s hybridisation was dead code for five documented versions.** More severe than the label drift above: [T1]’s own changelog records that `use_llm` was a silent no-op from its introduction in v3.5 through v4.2.1 – `SymbolicEngineWithLLM` [T2] was “never imported” / “never used” despite the flag existing in the public API – and was only actually wired up in v5.0. Any pre-v5.0 benchmark run requesting Variant 4’s seed/hybrid/fallback modes would have silently executed pure PySR instead (full detail and exact quotes in §3.5).
- **An unresolved reliability discrepancy is under active self-audit, not yet closed.** [E7] (`portfolio_variance_v3c`) exists specifically to reconcile, in its own words, “*The DeFi benchmark table (HypatiaX $R^2 = 1.000$ on Portfolio Variance) [vs.] The seed-sweep JSON (HypatiaX far- $R^2 = -882.9$ at the default seed 42)*” – i.e. a published near-perfect in-sample result for one test case coexists with a catastrophic far-extrapolation failure for the *same* case at the *same* seed. The notebook’s presence shows this is being actively investigated (seven audit sections culminating in a rerun/validate/export protocol) but, as retrieved, it is a live audit artefact rather than a closed, root-caused fix – exactly the catastrophic-extrapolation failure mode the paper’s own metrics (`compute_catastrophic_rate`, § “b) Description”) are designed to surface.
- **A second, distinct leakage class was found and fixed in the DeFi benchmark scripts.** [E6]’s own “Fix 7” comment (`hypatiaux_defi_benchmark_v3.py`, line 62) records that an earlier version computed ensemble uncertainty as `std(pred - y_test)` – i.e. using *test-set* residuals to weight the ensemble – and replaces it with a training-residual-only estimate. This is a different leakage mechanism from the prompt-embedded GT-LEAK bug class recorded in `core/` (§4 above), found in a different part of the codebase (benchmark ensembling, not LLM prompting), reinforcing that leakage has been a *recurring* category of bug across the project rather than a single fixed incident, and that – as with GT-LEAK – no automated check prevents a third occurrence.

6 e) Update – What Changed in This Revision

This section is a compact delta against the earlier `core/+experiments/` report, for a reader who has already seen it.

Area	Finding	Ref.
core/ (Variants 1–3)	Unchanged – identical line counts to the prior report; no functional edits detected.	[C1] -- [C6]
tools/symbolic/	Previously unread. Houses [T1] (HybridDiscoverySystem, v5.4 wrapper) and [T2] (symbolic_engine.py, whose SymbolicEngineWithLLM class is the <i>true</i> implementer of Variant 4’s four LLM/PySR modes, not [T1] – corrected mid-revision, see §3.4/§3.5), plus the template-based (Variant 5, [T3]) and template-free ([T4]) physics fallback.	[T1] -- [T4]
tools/validation/	Previously unread. A four-layer, weighted expression-acceptance pipeline (85/100 threshold), decoupled from Variants 1–3’s R^2 gates.	[T5] -- [T8]
tools/utils/	Previously unread. PCA-directed OOD train/test split, now the sole split in the _pca benchmark variants.	[U1]
experiments/benchmarks/	Four new/renamed files: an extracted extrapolation-ladder module, PCA-split counterparts of the v3/v4 DeFi benchmarks, and a self-audit notebook exposing an open in-sample-vs-far-extrapolation contradiction.	[E5] -- [E7]
protocols/	New top-level package, not requested; test-case data generators only, not analysed further.	(context only)

Net effect on the report’s earlier conclusions: the **four-variant** picture in the original “(b) Description” undercounted the pipeline by one production-relevant strategy (Variant 5) and oversimplified Variant 4 (one mechanism → four selectable modes); the **Limitations** in §4 gain four new, source-grounded entries (version drift, a five-version dead hybridisation path, an open reliability discrepancy, a second leakage class) that were not visible from core//experiments/ alone; and one of the original **Refactorization** recommendations below (“Promote ground-truth-leakage checks from comments to code”) turns out to be *partially* already addressed – [T5] -- [T8] is exactly this kind of promoted, code-level check, just scoped to expression validity in general rather than to the specific GT-LEAK prompt bug. **One self-correction within this revision:** an earlier draft of this section attributed Variant 4’s four-mode decision logic to [T1] (HybridDiscoverySystem); direct inspection of [T2] (symbolic_engine.py) showed the logic actually lives in SymbolicEngineWithLLM, with [T1] acting only as an orchestrating wrapper – corrected throughout §3–§6 above.

7 f) Refactorization

The codebase’s own comments already flag several of the refactoring opportunities below; this section consolidates and extends them based on direct inspection of core/, experiments/ and, in this revision, tools/.

7.1 Consolidate the duplicated API helper

_create_message_deterministic() appears near-verbatim in [C1], [C2] and [C4] (confirmed by direct comparison of all three definitions). The in-code comment explicitly acknowledges this is a deliberate

“per-file copies rather than cross-module imports” convention. **Recommendation:** extract to a single `hypatiaux/core/llm_client.py` module and import it in all three call sites; retain the current retry-on-deprecated-temperature behaviour verbatim so no functional change occurs. This removes the drift risk noted in §4 without altering any decision logic.

7.2 Unify the ensemble-weighting strategies

Introduce a single `hypatiaux/core/ensembling.py` exposing named strategies (`r2_share`, `clamped_r2_share`, `precision_weighted`) corresponding to Variants 1–3’s current inline formulas, selected by a `strategy=` parameter rather than being copy-pasted per hybrid class. This turns an implicit, undocumented inconsistency (§4, *Limitations*) into an explicit, testable choice, and enables a controlled ablation of ensembling strategy independent of domain (DeFi vs. all-domains).

7.3 Extract the shared `hybrid_predict` skeleton

`EnhancedHybridSystemDeFi.hybrid_predict [C1]` and `HybridSystemAllDomains.hybrid_predict [C2]` implement the same four-stage skeleton (generate → evaluate → train NN → decide) with different constants and an extra symbolic-refit stage in [C1]. **Recommendation:** factor a `BaseHybridSystem` class in `hypatiaux/core/generation/` with template-method hooks (`_refine_symbolic()`, `_decision_thresholds()`, `_domain_override()`), and have both concrete classes subclass it. This would make the threshold values called out in §4 (0.85/0.05, 0.90) into named, overridable class attributes instead of inline literals, directly enabling the sensitivity analysis that limitation currently lacks.

7.4 Centralise magic numbers following the `adaptive_config.py` pattern

`hypatiaux/core/training/adaptive_config.py` already demonstrates the target pattern the rest of `core/` should follow: a single frozen dataclass built from observable signals, replacing scattered literals with one documented “signal → parameter” table. The decision-gate thresholds in [C1]/[C2] (§4) are the clearest remaining candidates for migration into an analogous `DecisionThresholds` config object.

7.5 Promote ground-truth-leakage checks from comments to code

The `FIX GT-LEAK / FIX GT-LEAK-2` history (§4) shows this bug class has recurred at least twice across different prompt-construction functions. **Recommendation:** add a small static check – e.g. asserting the rendered prompt string does not contain any token from the case’s `ground_truth/expected_form` meta-data – run once in CI (`ci_pipeline_check.yml`) over every prompt-construction path in `core/`, rather than relying on manual code review to catch a recurrence.

7.6 Isolate LLM-generated code execution

`_safe_exec_formula` and `execute_python_code_get_predictions` both `exec()` raw LLM text in-process. **Recommendation:** route formula evaluation through a single shared, sandboxed evaluator (e.g. AST-validated against an allow-list of numpy calls before `exec`, or executed in a subprocess/resource-limited worker), replacing the two independent namespace-construction blocks currently duplicated between [C1] and [C3].

7.7 Compose the validation pipeline into the decision gate, not just alongside it

As §3.4 notes, `EnsembleValidator [T8]` and Variants 1–3’s R^2 gates currently run as two independent passes, so a formula can be selected by `hybrid_predict` and separately fail the 85/100 acceptance threshold with no interaction between the two outcomes. **Recommendation:** thread the validator’s score into `hybrid_predict`’s decision (e.g. as a multiplicative penalty on r^2_{fit} , or a hard veto below some validator

score) so an expression that is symbolically or dimensionally invalid cannot win the `fitted_llm/llm` branch purely on training R^2 .

7.8 Record the engine version in its own output, not just its docstring

[T1]’s docstring is versioned (v5.4, with a five-entry RC-* changelog) but its `save_results()` output and the caller’s ([E2]) results dictionary are not – the version currently lives only in prose comments that have already drifted out of sync once (§4). **Recommendation:** expose `HybridDiscoverySystem.VERSION` (already defined as a class attribute, [T1] line 249) in every results payload the engine returns, and have [E2] read that attribute for its own "engine" label instead of a hard-coded string, so result artefacts are self-describing and the v5.1/v5.4 mismatch found in this revision cannot recur silently.

7.9 Reduce experiment-script duplication via composition, not copy-paste

`expl_five_system.py` already states the intended pattern in its own docstring – “*Nothing in this file reimplements method logic...reused, not reimplemented*” – confirming the project’s own stated direction. **Recommendation:** apply that same discipline retroactively to the other `run_*_benchmark.py` scripts in `experiments/benchmarks/`, several of which (per file sizes in §2, e.g. `hypatix_defi_benchmark_v3.py` and `_v4.py` both at 2,322 lines) appear to be near-duplicate versioned copies rather than a single parameterised script – a strong candidate for collapsing into one script with a `--benchmark-version` flag, reducing the maintenance surface for future `FIX-*` patches to one location instead of two.

8 Summary

Variant	Combination rule	Domain	Location
1. Enhanced Hybrid DeFi	threshold gate + scipy refit	DeFi	[C1]
2. Hybrid All-Domains	tie-favours-symbolic gate + force_llm override	Physics/Chem/Bio	[C2]
3. Ensemble example	inverse-residual-std weighting	DeFi (demo)	[C3]
4. LLM-prior / PySR family	4 modes: none/seed/hybrid/fallback, thresholds 0.5/0.90/0.95	Nguyen/Feynman	[T2] (via [T1]/[E1])
5. Template physics fallback	template-library or template-free structure fit	Biology/Chem/Eng.	[T3]/[T4]
<i>Cross-cutting, applies to any variant’s output:</i>			
Expression validation	4-layer weighted score, 85/100 threshold	all domains	[T5] -- [T8]
PCA-directed OOD split	held-out region beyond PC1 range	DeFi	[U1]/[E6]

This report was produced entirely from static reading of the source files listed in §2 (this revision: the full repository tree, fetched as one tarball from `codelead.github.com` at master, plus the individual files listed in §2.2–§2.4); no code in the repository was executed during its preparation.